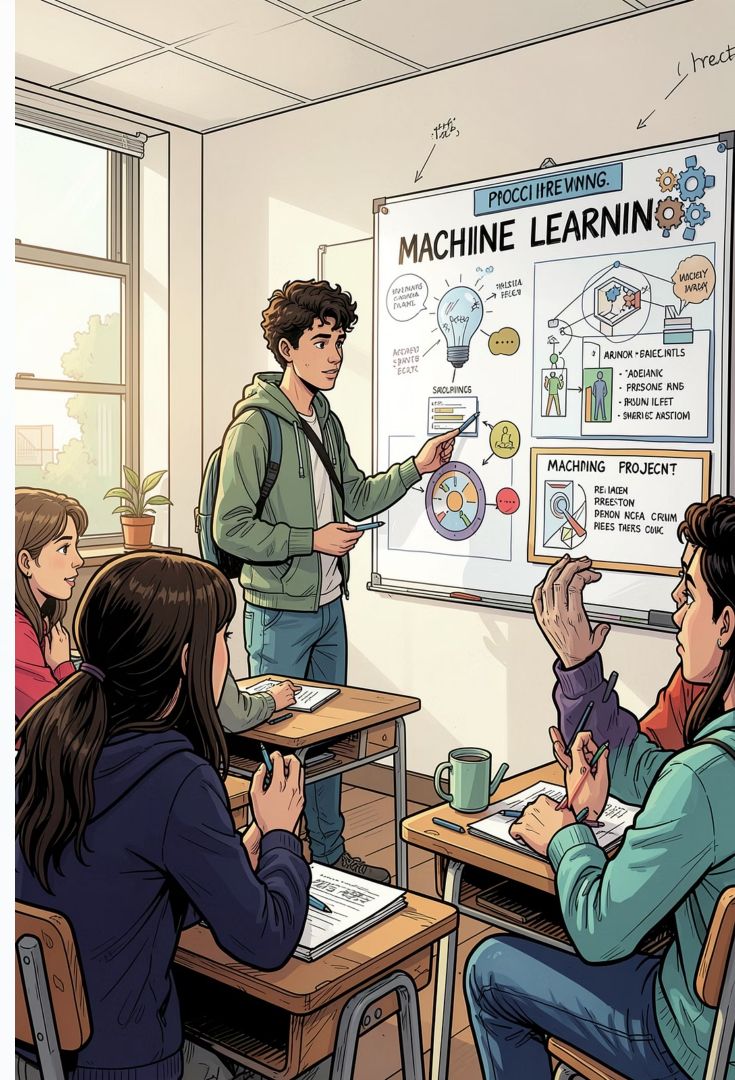


Netflix Movie Recommendation System

Item-based Collaborative Filtering, SVD matrix factorization, and a hybrid model — Machine Learning project comparing collaborative filtering, matrix factorization, and hybrid recommendation techniques.

- Team Member 1: Sparsh Agrawal (24113126)
- Team Member 2: Ritesh Choudhary (24113105)



Problem Overview



Why recommender systems matter: help users find relevant movies in huge catalogs, increase engagement, and reduce decision fatigue.

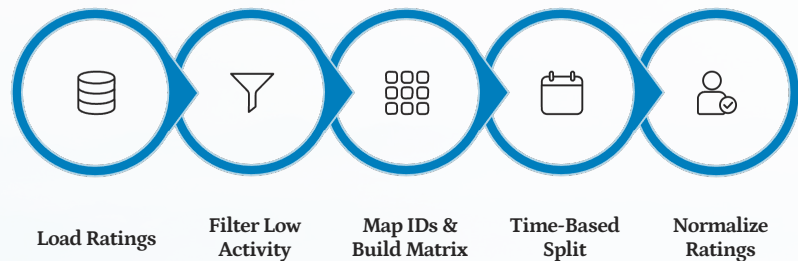
Key challenges: very sparse user–item matrices, popularity bias, cold-start users/movies, and noisy ratings.

Project objective: compare Item-based CF, SVD, and a Hybrid model on the Netflix dataset; select a final model based on accuracy and ranking metrics.

Dataset & Preprocessing

Source: Netflix Prize dataset.

- Full: >100M ratings (original dataset)
- Modeling subset: 50,000 users, 7,291 movies, 43.2M ratings
- SVD subset: 8,000 sampled users, ~6.9M ratings
- Filtering: remove users/movies with < 500 ratings; timestamp-based train/test split (80/20)

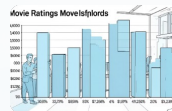


Normalization: subtract global mean and per-user bias before matrix factorization; store mappings for candidate generation.

Key Insights from Exploratory Analysis

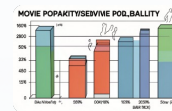
Quick facts:

- Mean rating = **3.52** (positive skew)
- Top movies account for a large share of interactions
- Small portion of users produce most ratings (long tail)
- Matrix sparsity $\approx$ **88.1%**



Rating Distribution

Most ratings cluster around 3–4 stars.

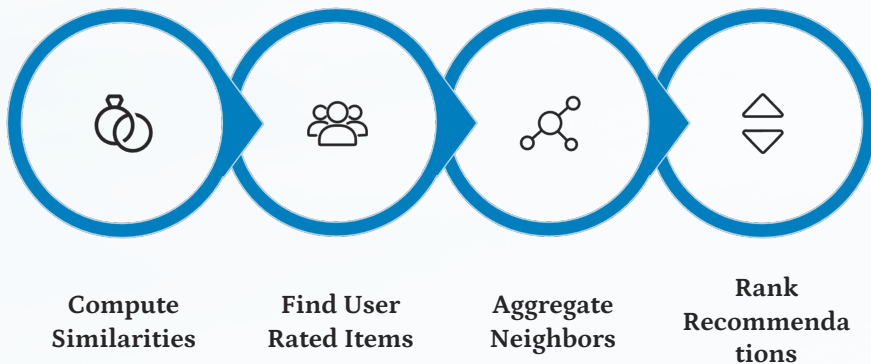


Movie Popularity

Few movies get most views; many get very few.

Recommendation generation: for each user, score candidate items by summing similarity $\times$ user rating (normalize by similarity sum).

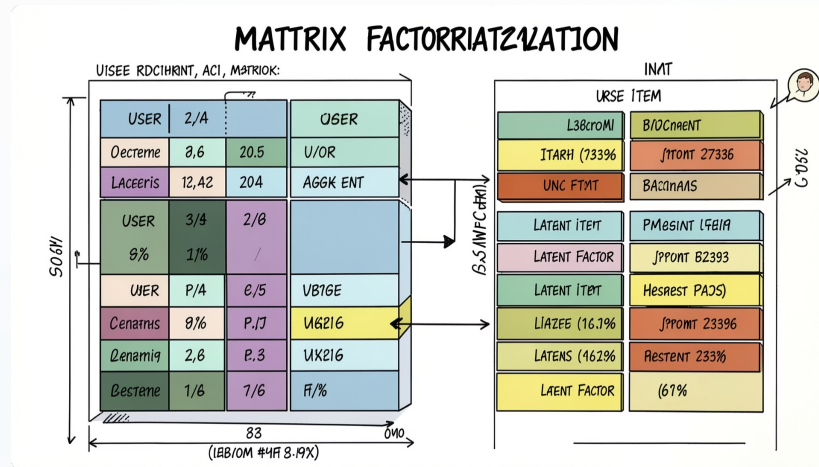
Limitations: suffers when items have few co-ratings; popularity bias; limited ability to model hidden preferences.



SVD Matrix Factorization

Idea in simple terms: approximate the user-item matrix by two low-rank matrices (user factors and item factors). Each factor dimension encodes latent preferences (e.g., genre taste, popularity).

- Objective: minimize regularized squared error on observed ratings
- Important hyperparameters: number of factors (k), learning rate, regularization λ , number of epochs



Advantages: captures latent structure, handles sparsity better, can model complex preferences.

Limitations: training cost, needs tuning, may overfit without regularization.

Hybrid Recommendation Approach

Why combine? Item CF provides interpretable neighborhood signals; SVD captures latent patterns. Hybrid balances both strengths and reduces weaknesses.

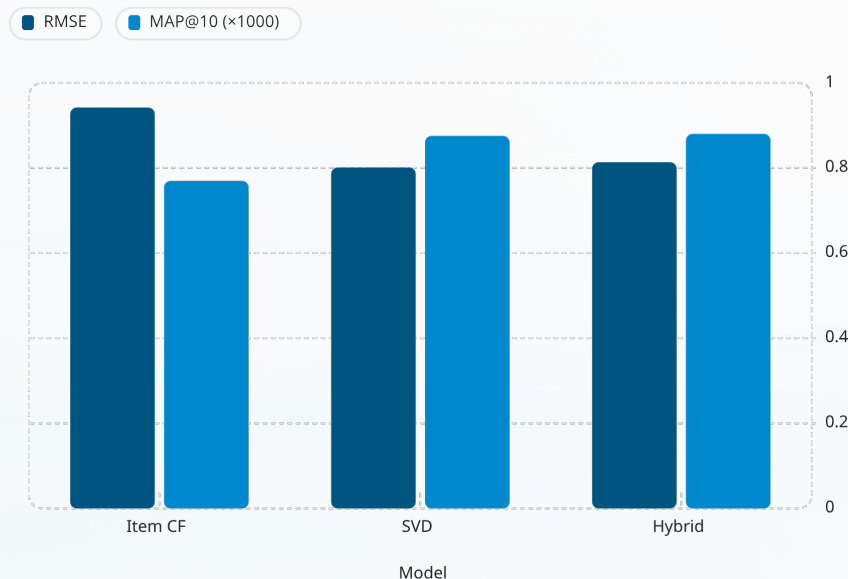
Score combination: Final Score = $(0.7 \times \text{SVD Score}) + (0.3 \times \text{Item CF Score})$

Normalization & weighting: before combining, both model scores are min-max normalized per user to $[0,1]$. Weights chosen after cross-validation to favor SVD's accuracy while keeping CF diversity.



Final ranking uses combined score to produce top-10 lists for each user.

Experimental Results & Model Comparison



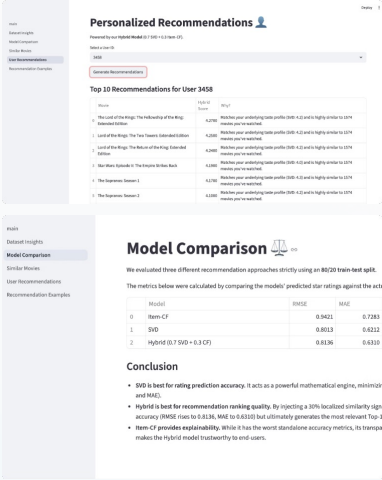
RMSE: lower is better. MAP@10: higher is better. SVD gives best RMSE and strong MAP. Hybrid slightly trails SVD on RMSE but improves MAP@10 over SVD.

Hybrid achieved the highest MAP@10 (0.8800), indicating better recommendation ranking quality.

Metric	Values
Item CF RMSE / MAP@10	0.9421 / 0.7697
SVD RMSE / MAP@10	0.8013 / 0.8754
Hybrid RMSE / MAP@10	0.8136 / 0.8800

Why pick Hybrid? It slightly improves ranking quality (MAP@10) while retaining SVD's accuracy—giving better top-list relevance.

User Example, Conclusion & Future Work



Example — User 11089:

Top-10 recommendations :

Model	Sample Top-10
Item CF	Popular action titles, similar to user's rated items
SVD	Mix of popular and niche titles aligned with latent taste
Hybrid	Balanced list: relevant niche picks + well-rated popular titles

Conclusions: SVD achieves best RMSE; Hybrid improves ranking (MAP@10) and yields more balanced top-k lists. Item CF remains useful for explainability.

Future work:

- Neural Collaborative Filtering to learn non-linear interactions
- Include content-based features (genres, cast) to handle cold-starts
- Real-time API deployment and A/B testing for online evaluation



Project artifacts: code, preprocessing scripts, trained models, and evaluation notebooks available on request.

